

claude-windows / 2ba49afa-99f4-41f2-86a2-27a289d432f3

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\2ba49afa-99f4-41f2-86a2-27a289d432f3\subagents\workflows\wf_5d655769-ba4\agent-ab7464429d5a3a8f3.jsonl`  
- SHA-256: `15f19200788dd2fda2d55847872c7ac2d26fd1b69ca85b928969ac4b2141774a`  
- Source modified: `2026-06-21T17:19:40+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_5d655769-ba4`  
- session_id: `2ba49afa-99f4-41f2-86a2-27a289d432f3`
```

Transcript

1. user (2026-06-21T17:17:44.234Z)

Review A9 for CORRECTNESS. A9 (tracker FRIENDLY_AND_MULTILINGUAL \$7): localized tap-only filter controls so a non-English-typing user can filter without typing the English query syntax. Files under C:/Users/avidu/Projects/khelsutra/web/src: NEW components/TapFilters.tsx (exports TAP_FILTERS data + the component); App.tsx renders <TapFilters onApply={onApplyQuery}/> below <QueryBar/>; catalogs en/kn/hi/es/da/id gained a tap.* block (title/longest/shortest/over10/over20/top5/top10). Each control builds a ParsedQuery DIRECTLY (no parsing) and calls the shared onApply(q,"example"). A test pins tap==type (parseQuery(f.english) deep-equals f.parts). 173 web tests green; tsc+build clean; browser-verified (tap "Over 10 seconds" ? 3 rallies; labels localize).
Read C:/Users/avidu/Projects/khelsutra/web/src/components/TapFilters.tsx,
C:/Users/avidu/Projects/khelsutra/web/src/lib/query.ts (parseQuery + runQuery),
C:/Users/avidu/Projects/khelsutra/web/src/App.tsx (onApplyQuery).
1. tap==type INVARIANT: for EACH TAP_FILTER, does parseQuery(english) truly produce the same {filters,sort,limit} as parts? Manually trace "top 5" (does parseQuery set sort duration desc + limit 5?), "over 10s", "longest first". Any drift the test would miss (e.g. parseQuery adds a sort the parts omit)?
2. The constructed ParsedQuery: filters/sort/limit/unavailable=[]/recognized=true ? does App.onApplyQuery handle it identically to a typed query (runQuery + sel.apply + logging)? The raw is "tap:tap.longest" ? any issue with that raw string in logs/parseTrace?
3. Are the controls length-led + fully-available (no shot-count, no unavailable tokens) per the doc?
Be precise. Empty findings if correct.

2. assistant (2026-06-21T17:17:47.835Z)

I'll review A9 for correctness. Let me start by reading the relevant files.

3. user (2026-06-21T17:17:48.789Z)

```
1      // Localized tap-only filter controls (tracker A9). The query bar PARSES ENGLISH text (a documented  
2      // limit) ? so a Kannada/Hindi/? user who can't type English needs to filter by TAPPING. Each control  
3      // builds the SAME ParsedQuery the parser would for its English equivalent ? directly, no English  
4      // round-trip ? and applies it through the shared onApply path. A test pins tap == type so a control  
5      // can never silently drift from the parser. Length-led + fully-available only (never shot-count).  
6      import { useTranslation } from "react-i18next";  
7      import type { ParsedQuery } from "../lib/query";  
8  
9      interface TapFilter {  
10         /** i18n label key (localized) ? the user sees this, not the English query. */
```

```

11     key: string;
12     /** The equivalent typed query; the tap MUST select identically to typing this (pinned
by a test). */
13     english: string;
14     parts: Pick<ParsedQuery, "filters" | "sort" | "limit">;
15   }
16
17   export const TAP_FILTERS: TapFilter[] = [
18     { key: "tap.longest", english: "longest first", parts: { filters: [], sort: { key:
"duration", dir: "desc" } } },
19     { key: "tap.shortest", english: "shortest first", parts: { filters: [], sort: { key:
"duration", dir: "asc" } } },
20     { key: "tap.over10", english: "over 10s", parts: { filters: [{ field: "duration", op:
">", value: 10 } ] } },
21     { key: "tap.over20", english: "over 20s", parts: { filters: [{ field: "duration", op:
">", value: 20 } ] } },
22     { key: "tap.top5", english: "top 5", parts: { filters: [], sort: { key: "duration",
dir: "desc" }, limit: 5 } },
23     { key: "tap.top10", english: "top 10", parts: { filters: [], sort: { key: "duration",
dir: "desc" }, limit: 10 } },
24   ];
25
26   interface Props {
27     /** Same handler the QueryBar uses ? App runs runQuery + writes the shared selection.
*/
28     onApply: (q: ParsedQuery, source: "typed" | "example") => void;
29   }
30
31   export function TapFilters({ onApply }: Props) {
32     const { t } = useTranslation();
33     // ?44px tap targets (A4) so the no-type path is comfortable on a phone.
34     const BTN = "inline-flex min-h-11 items-center rounded-full border border-black/15
px-4 text-sm hover:border-green";
35
36     return (
37       <div className="mt-3" data-testid="tap-filters">
38         <span className="text-xs text-ink/40">{t("tap.title")}</span>
39         <div className="mt-1.5 flex flex-wrap gap-2">
40           {TAP_FILTERS.map((f) => (
41             <button
42               key={f.key}
43               type="button"
44               className={BTN}
45               onClick={() =>
46                 onApply(
47                   { raw: `tap:${f.key}`, filters: f.parts.filters, sort: f.parts.sort,
limit: f.parts.limit, unavailable: [], recognized: true },
48                   "example",
49                 )
50             )}
51           >
52             {t(f.key)}
53           </button>
54         )]}
55       </div>
56     </div>
57   );
58 }
59

```

4. user (2026-06-21T17:17:49.262Z)

```

1     // Deterministic, dependency-free query compiler for the rally picker
(PER_RALLY_SELECTION.md
2     // D2/D5). NO LLM, NO network ? a tiny regex grammar over the ONLY honest, queryable

```

```

axes the
3      // engine returns today: rally LENGTH (duration), ORDER (start_time / duration /
shot_count),
4      // COUNT (top/first N), and SHOT COUNT (metadata.shot_count).
5      //
6      // Everything else a user might type ? shot TYPE (smash/clear/drop?), PLAYER, SCORE ?
has no
7      // detector and no column, so we FLAG it as "not available yet" and NEVER fabricate a
result.
8      // Parsing is pure and unit-tested; the QueryBar React component is a thin shell over
it.
9      import type { RallySegment } from "../api/types";
10
11      export type CompareOp = ">" | ">=" | "<" | "<=" | "=";
12      export type FilterField = "duration" | "shot_count";
13      /** One ANDed predicate over a rally field (e.g. duration > 10s). */
14      export interface Filter {
15          field: FilterField;
16          op: CompareOp;
17          value: number; // seconds for duration; a raw count for shot_count
18      }
19
20      export type SortKey = "duration" | "start_time" | "shot_count";
21      export interface Sort {
22          key: SortKey;
23          dir: "asc" | "desc";
24      }
25
26      /** A recognized-but-unsupported concept ? surfaced as a greyed "not available yet"
hint. */
27      export type UnavailableKind = "shot-type" | "player" | "score";
28      export interface Unavailable {
29          token: string;
30          kind: UnavailableKind;
31      }
32
33      export interface ParsedQuery {
34          raw: string;
35          /** ANDed together. */
36          filters: Filter[];
37          sort?: Sort;
38          /** "top N" / "first N" / "N longest". */
39          limit?: number;
40          /** Concepts we understood but cannot honour (shot-type / player / score). */
41          unavailable: Unavailable[];
42          /** True when at least one actionable clause (filter | sort | limit) was understood.
*/
43          recognized: boolean;
44      }
45
46      export interface QueryResult {
47          /** ALL rallies in display order (sorted if a sort was given, else input order). */
48          ordered: RallySegment[];
49          /** Ids matching the filters (then limited), in display order. */
50          selectedIds: number[];
51      }
52
53      // ?? grammar fragments ?????????????????????????????????????????????????????????
54      // `(?!\d)` stops a greedy digit-run from backtracking and leaving stray trailing digits
55      // (so "over 10 smashes" can't degrade to "over 1" + "0 smashes").
56      const NUM = "(\\d+(?:\\.\\d+)?)(?!\\d)";
57      // Time / "shots" unit tokens. "shots?" is FIRST so it isn't eaten by the bare "s";
"mins?"
58      // already covers "min" (no redundant bare "min"); \b stops "m" matching "minutes" etc.
59      const UNIT_INNER = "shots?|seconds?|secs?|minutes?|mins?|m|s";

```

```

60     const UNIT = `(?:\\s*(${UNIT_INNER})\\b)?`;
61
62     // Shot-TYPE / player / score nouns ? concepts with NO detector and NO column. A number
trailing
63     // one of these must NEVER become a real filter (honesty: never reroute an unsupported
concept
64     // onto the duration axis ? PER_RALLY_SELECTION.md §3/§4.3). These stems are shared with
the
65     // "not available yet" flagging below so the two can never drift apart.
66     const SHOT_TYPE = "smash(?:es)?|clears?|drops?|dropshots?|net
?shots?|drives?|lifts?|serves?|kills?|taps?|pushes?|blocks?|slices?";
67     const PLAYER = "players?|opponents?|servers?|receivers?";
68     const SCORE = "points?|score|deuce|game point|set point|match point";
69     const UNAVAIL_WORD = `(?:${SHOT_TYPE}|${PLAYER}|${SCORE})`;
70     // Voids a comparator clause whose number is immediately followed by an unsupported
noun.
71     const NOT_UNAVAIL = `(!\\s*${UNAVAIL_WORD}\\b)`;
72
73     interface OpPattern {
74         op: CompareOp;
75         re: RegExp;
76         /** Which capture group holds the number / the unit. */
77         numAt: number;
78         unitAt: number;
79     }
80
81     // Leading comparator ("over 10s"). The "no more than" / "at least" guards come BEFORE
their
82     // bare counterparts ("more than" / "less than") so the negated phrase is consumed
first.
83     function lead(alt: string, op: CompareOp): OpPattern {
84         return { op, re: new RegExp(`(?:${alt})\\s*${NUM}${UNIT}${NOT_UNAVAIL}`, "g"), numAt:
1, unitAt: 2 };
85     }
86     const LEADING: OpPattern[] = [
87         lead("no more than|at most|maximum(?: of)?|max|up to|<=", "<="),
88         lead("no less than|no fewer than|at least|minimum(?: of)?|min|>=", ">="),
89         lead("exactly|equal to|equals", "="),
90         lead("over|longer than|more than|greater than|above|bigger than|>", ">"),
91         lead("under|shorter than|less than|fewer than|below|smaller than|<", "<"),
92     ];
93
94     // Trailing comparator. The unit may sit BEFORE the operator ("10s+", "8 shots or more")
or the
95     // "shots" noun AFTER it ("10+ shots") ? capture both sides and use whichever is
present.
96     // NOT_UNAVAIL guards the same way ("5+ smashes" must not become duration ? 5).
97     interface TrailPattern {
98         op: CompareOp;
99         re: RegExp;
100     }
101     const POST_NOUN = "(?:\\s*(shots?)\\b)?"
102     const TRAILING: TrailPattern[] = [
103         { op: ">=", re: new RegExp(`${NUM}${UNIT}\\s*(?:\\+|or
(?:more|longer|over|above|greater)) ${POST_NOUN} ${NOT_UNAVAIL}`, "g") },
104         { op: "<=", re: new RegExp(`${NUM}${UNIT}\\s*or
(?:less|fewer|shorter|under|below) ${POST_NOUN} ${NOT_UNAVAIL}`, "g") },
105     ];
106
107     function toSeconds(n: number, unit?: string): number {
108         return unit && /^m/.test(unit) ? n * 60 : n; // m/min/minute(s) ? seconds; else as-is
109     }
110
111     function makeFilter(numStr: string, unit: string | undefined, op: CompareOp): Filter {
112         const n = Number(numStr);

```

```

113     const isShots = !!unit && /^shots?$/i.test(unit);
114     return isShots
115     ? { field: "shot_count", op, value: n }
116     : { field: "duration", op, value: toSeconds(n, unit) };
117 }
118
119     const SORT_PATTERNS: Array<[RegExp, Sort]> = [
120         [/b(?:longest(?: first)?|longer first|by(?:duration|length)|duration desc)\b/, {
key: "duration", dir: "desc" }],
121         [/b(?:shortest(?: first)?|shorter first|duration asc)\b/, { key: "duration", dir:
"asc" }],
122         [/b(?:newest(?: first)?|latest|most recent|recent first)\b/, { key: "start_time",
dir: "desc" }],
123         [/b(?:oldest(?: first)?|earliest|chronological|in order|by time)\b/, { key:
"start_time", dir: "asc" }],
124         [/b(?:most shots(?: first)?|highest shots?|by shots?|by shot count|shot count
desc)\b/, { key: "shot_count", dir: "desc" }],
125         [/b(?:fewest shots(?: first)?|least shots?|lowest shots?|shot count asc)\b/, { key:
"shot_count", dir: "asc" }],
126     ];
127
128     // Order before kind so a token claimed by an earlier kind isn't double-counted. The
stems are the
129     // same SHOT_TYPE/PLAYER/SCORE used by NOT_UNAVAIL, so "flag it" and "don't filter on
it" stay in sync.
130     const UNAVAILABLE_PATTERNS: Array<[UnavailableKind, RegExp]> = [
131         ["shot-type", new RegExp(`\\b(${SHOT_TYPE})\\b`, "g")],
132         ["player", new RegExp(`\\b(${PLAYER})\\b`, "g")],
133         ["score", new RegExp(`\\b(${SCORE})\\b`, "g")],
134     ];
135
136     // A magnitude that carries a time/"shots" unit is NOT an ordinal count ("first 30
seconds" ? the
137     // first 30 rallies). This blocks the limit regexes from misreading such a number as a
count.
138     const NOT_UNIT = `(?!\\s*(?:${UNIT_INNER})\\b)`;
139
140     // ?? public API ?????????????????????????????????????????????????????????????
141
142     export function parseQuery(raw: string): ParsedQuery {
143         const normalized = raw
144             .toLowerCase()
145             .replace(/>/g, ">=")
146             .replace(/</g, "<=")
147             .replace(/(\d),(?=\d{3}\b)/g, "$1") // strip thousands separators ("1,000 shots" ?
"1000 shots")
148             .replace(/,/g, " ") // any remaining comma is a clause separator
149             .replace(/\s+/g, " ")
150             .trim();
151
152         const filters: Filter[] = [];
153         let working = normalized;
154
155         // Extract comparator clauses, blanking each match so later passes can't re-read its
number.
156         for (const pat of LEADING) {
157             working = working.replace(pat.re, (full, ...g) => {
158                 filters.push(makeFilter(g[pat.numAt - 1], g[pat.unitAt - 1], pat.op));
159                 return " ".repeat(full.length);
160             });
161         }
162         for (const pat of TRAILING) {
163             working = working.replace(pat.re, (full, num, preUnit, postNoun) => {
164                 filters.push(makeFilter(num, preUnit ?? postNoun, pat.op));
165                 return " ".repeat(full.length);

```

```

166     });
167 }
168
169 // Explicit sort (runs before limit so "top 15 longest" keeps the longest?desc
intent).
170 let sort: Sort | undefined;
171 for (const [re, s] of SORT_PATTERNS) {
172     if (re.test(working)) {
173         sort = s;
174         break;
175     }
176 }
177
178 // Limit ("top/best N", "first N", "last N", "N longest/shortest", "longest/shortest
N"). A number
179 // carrying a unit (\b + NOT_UNIT) is NOT a count, so "first 30 seconds" / "longest 3
shots" don't
180 // become counts. `sortFromLimit` tracks a sort implied SOLELY by the limit, so if the
limit turns
181 // out invalid (e.g. "top 0") we drop that phantom sort too and the whole query reads
as a no-op.
182 let limit: number | undefined;
183 let sortFromLimit = false;
184 let m: RegExpMatchArray | null;
185 if (
186     (m = working.match(new RegExp(`\\b(\\d+)\\b${NOT_UNIT}\\s+(longest|shortest)\\b`)))
||
187     (m = working.match(new RegExp(`\\b(longest|shortest)\\s+(\\d+)\\b${NOT_UNIT}`)))
188 ) {
189     const num = /^\\d/.test(m[1]) ? m[1] : m[2];
190     const word = /^\\d/.test(m[1]) ? m[2] : m[1];
191     limit = parseInt(num, 10);
192     if (!sort) {
193         sort = { key: "duration", dir: word === "longest" ? "desc" : "asc" };
194         sortFromLimit = true;
195     }
196 } else if ((m = working.match(new RegExp(`\\b(?:top|best)\\s+(\\d+)\\b${NOT_UNIT}`)))) {
197     limit = parseInt(m[1], 10);
198     if (!sort) {
199         sort = { key: "duration", dir: "desc" }; // "top N" ? best = longest
200         sortFromLimit = true;
201     }
202 } else if ((m = working.match(new
RegExp(`\\b(?:first|earliest)\\s+(\\d+)\\b${NOT_UNIT}`)))) {
203     limit = parseInt(m[1], 10);
204     if (!sort) {
205         sort = { key: "start_time", dir: "asc" };
206         sortFromLimit = true;
207     }
208 } else if ((m = working.match(new RegExp(`\\b(?:last|latest|newest|most
recent)\\s+(\\d+)\\b${NOT_UNIT}`)))) {
209     limit = parseInt(m[1], 10);
210     if (!sort) {
211         sort = { key: "start_time", dir: "desc" };
212         sortFromLimit = true;
213     }
214 }
215
216 // A limit must be a positive integer. Reject "top 0" (which would otherwise slice to
an EMPTY
217 // selection and silently wipe the user's curation) and clamp absurd magnitudes so the
pill never
218 // renders scientific notation.
219 if (limit !== null) {

```

```

220     if (!Number.isInteger(limit) || limit < 1) {
221         limit = undefined;
222         if (sortFromLimit) sort = undefined;
223     } else if (limit > Number.MAX_SAFE_INTEGER) {
224         limit = Number.MAX_SAFE_INTEGER;
225     }
226 }
227
228 // Unavailable concepts (scanned on the normalized text; deduped, first kind wins).
229 const unavailable: Unavailable[] = [];
230 const seen = new Set<string>();
231 for (const [kind, re] of UNAVAILABLE_PATTERNS) {
232     for (const mm of normalized.matchAll(re)) {
233         const token = mm[1];
234         if (!seen.has(token)) {
235             seen.add(token);
236             unavailable.push({ token, kind });
237         }
238     }
239 }
240
241 return {
242     raw,
243     filters,
244     sort,
245     limit,
246     unavailable,
247     recognized: filters.length > 0 || sort !== null || limit !== null,
248 };
249 }
250
251 function rallyValue(r: RallySegment, key: SortKey): number | undefined {
252     if (key === "duration") return r.duration;
253     if (key === "start_time") return r.start_time;
254     return r.metadata?.shot_count;
255 }
256
257 /** Stable sort; rallies missing the sort field always sort LAST (unknown ? smallest).
258 */
259 export function sortRallies(rallies: RallySegment[], sort: Sort): RallySegment[] {
260     return rallies
261         .map((r, i) => ({ r, i }))
262         .sort((a, b) => {
263             const av = rallyValue(a.r, sort.key);
264             const bv = rallyValue(b.r, sort.key);
265             if (av == null && bv == null) return a.i - b.i;
266             if (av == null) return 1;
267             if (bv == null) return -1;
268             if (av !== bv) return sort.dir === "asc" ? av - bv : bv - av;
269             return a.i - b.i; // tie ? preserve input order
270         })
271         .map((x) => x.r);
272 }
273
274 function matchesFilter(r: RallySegment, f: Filter): boolean {
275     const v = f.field === "duration" ? r.duration : r.metadata?.shot_count;
276     if (v == null) return false; // a rally without shot_count can't satisfy a shot filter
277     switch (f.op) {
278         case ">":
279             return v > f.value;
280         case ">=":
281             return v >= f.value;
282         case "<":
283             return v < f.value;
284         case "<=":

```

```

284         return v <= f.value;
285     case "=":
286         return v === f.value;
287     default:
288         return false;
289     }
290 }
291
292 /** Apply a parsed query to the rallies: ordered display list + the ids it selects.
Pure.
293 * An UNRECOGNIZED query (nothing understood) selects nothing and leaves order untouched
?
294 * so garbage input can never silently wipe a user's manual curation. The App also
guards on
295 * `q.recognized` before applying, so an empty box is a no-op end to end. */
296 export function runQuery(rallies: RallySegment[], q: ParsedQuery): QueryResult {
297     if (!q.recognized) return { ordered: rallies.slice(), selectedIds: [] };
298     const ordered = q.sort ? sortRallies(rallies, q.sort) : rallies.slice();
299     let matched = ordered.filter((r) => q.filters.every((f) => matchesFilter(r, f)));
300     if (q.limit !== null) matched = matched.slice(0, q.limit);
301     return { ordered, selectedIds: matched.map((r) => r.id) };
302 }
303
304 // ?? human-readable labels (shared by the QueryBar pill and the tests) ?????????
305 const OP_SYMBOL: Record<CompareOp, string> = { ">": ">", ">=": "?", "<": "<", "<=": "?",
"=": "=" };
306
307 export function filterLabel(f: Filter): string {
308     if (f.field === "duration") return `length ${OP_SYMBOL[f.op]} ${f.value}s`;
309     return `shots ${OP_SYMBOL[f.op]} ${f.value}`;
310 }
311
312 export function sortLabel(s: Sort): string {
313     if (s.key === "duration") return s.dir === "desc" ? "longest first" : "shortest
first";
314     if (s.key === "start_time") return s.dir === "desc" ? "newest first" : "oldest first";
315     return s.dir === "desc" ? "most shots first" : "fewest shots first";
316 }
317
318 /** The count label, derived from the limit's actual sort so the pill can't contradict
itself
319 * ("first 10", not "top 10", when the sort is oldest-first). */
320 export function limitLabel(limit: number, sort?: Sort): string {
321     if (sort?.key === "start_time") return sort.dir === "asc" ? `first ${limit}` : `last
${limit}`;
322     if (sort?.key === "duration" && sort.dir === "asc") return `shortest ${limit}`;
323     if (sort?.key === "shot_count") return sort.dir === "asc" ? `fewest ${limit}` : `top
${limit} by shots`;
324     return `top ${limit}`; // duration desc, or no sort
325 }
326
327 /** Human clause labels for the parsed-pill (e.g. ["length > 10s", "longest first", "top
15"]). */
328 export function summarize(q: ParsedQuery): string[] {
329     const parts: string[] = [];
330     for (const f of q.filters) parts.push(filterLabel(f));
331     if (q.sort) parts.push(sortLabel(q.sort));
332     if (q.limit !== null) parts.push(limitLabel(q.limit, q.sort));
333     return parts;
334 }
335

```

5. user (2026-06-21T17:17:49.373Z)

```

1     import { useEffect, useMemo, useState } from "react";

```



```

2     import { useTranslation, Trans } from "react-i18next";
3     import { useRallySelection } from "../hooks/useRallySelection";
4     import { RallyGrid } from "../components/RallyGrid";
5     import { QueryBar } from "../components/QueryBar";
6     import { TapFilters } from "../components/TapFilters";
7     import { SelectionFooter } from "../components/SelectionFooter";
8     import { LanguageSwitcher } from "../components/LanguageSwitcher";
9     import { CompileResult, type BuildState } from "../components/CompileResult";
10    import { IngestPanel } from "../components/IngestPanel";
11    import { runQuery, type ParsedQuery } from "../lib/query";
12    import { floorRisk } from "../lib/floor";
13    import { reelFilename } from "../lib/format";
14    import { classifyError } from "../lib/errors";
15    import { errorText } from "../lib/errorText";
16    import { log, newCorrelationId } from "../lib/log";
17    import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
from "../api/client";
18    import type { EngineConfig, RallySegment } from "../api/types";
19
20    /** One compact, structured description of how a query was understood (for the trace
log). */
21    function parseTrace(q: ParsedQuery) {
22        return {
23            raw: q.raw,
24            recognized: q.recognized,
25            filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
26            sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
27            limit: q.limit ?? null,
28            unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
29        };
30    }
31
32    // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
33    // contract; only honest fields are surfaced. A real session loads from POST /api/query.
34    const DEMO_RALLIES: RallySegment[] = [
35        { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
36        { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
37        { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
38        { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
39        { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
40        { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
41    ];
42
43    type LoadState =
44        | { status: "demo" }
45        | { status: "loading" }
46        | { status: "loaded"; count: number }
47        | { status: "error"; error: string };
48
49    export default function App() {
50        const { t } = useTranslation();
51        // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,
52        // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
53        // without a reload (it also rewrites ?video= so a refresh is stable).
54        const [videoId, setVideoId] = useState<string | null>()
55        () => new URLSearchParams(window.location.search).get("video"),

```

```

56     );
57
58     const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
59     const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
60     const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
61     const [reelName, setReelName] = useState("highlights");
62     const [build, setBuild] = useState<BuildState>({ status: "idle" });
63
64     // Display order is separate from selection: a query can re-sort the cards while the
shared
65     // selection set (sel) stays the single source of truth for what lands in the reel.
66     const sel = useRallySelection(rallies, "all");
67
68     // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
69     useEffect(() => {
70         if (!videoId) return;
71         const cid = newCorrelationId();
72         log("info", "rallies.load_start", { cid, video_id: videoId });
73         // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
74         // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
75         setLoadState({ status: "loading" });
76         let alive = true;
77         listRallies(videoId, cid)
78             .then((rs) => {
79                 if (!alive) return;
80                 setRallies(rs);
81                 setOrder(rs);
82                 sel.apply(rs.map((r) => r.id));
83                 setLoadState({ status: "loaded", count: rs.length });
84                 log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
85             })
86             .catch((err) => {
87                 if (!alive) return;
88                 // Friendly, localized ? same shared mapping as build/ingest (A5): a load
failure on a slow or
89                 // down engine reads "is it running?", not a raw "ApiError: Network error
calling POST ?".
90                 const e = classifyError(err);
91                 setLoadState({ status: "error", error: errorText(t, e) });
92                 log("error", "rallies.load_failed", { cid, video_id: videoId, code: e.code,
detail: e.detail, error: String(err) });
93             });
94         return () => {
95             alive = false;
96         };
97         // videoId is stable for the page; sel.apply is a stable callback.
98         // eslint-disable-next-line react-hooks/exhaustive-deps
99     }, [videoId]);
100
101     // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
102     // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
103     const [cfg, setCfg] = useState<EngineConfig | null>(null);
104     useEffect(() => {
105         let alive = true;
106         getConfig()
107             .then((c) => alive && setCfg(c))
108             .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
109         return () => {
110             alive = false;
111         };
112     }, []);

```

```

113     const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
114
115     // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
116     const ordinalOf = useMemo(() => {
117         const m = new Map<number, number>();
118         [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
119         return (id: number) => m.get(id) ?? 0;
120     }, [rallies]);
121
122     // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
123     const onBuild = async () => {
124         if (build.status === "building") return;
125         const cid = newCorrelationId();
126         const filename = reelFilename(reelName);
127         const ids = [...sel.selected];
128         if (!videoId) {
129             log("info", "build.no_video", { cid, selected: ids.length });
130             setBuild({
131                 status: "error",
132                 error: t("build.noVideo"),
133             });
134             return;
135         }
136         setBuild({ status: "building" });
137         log("info", "build.start", {
138             cid,
139             video_id: videoId,
140             filename,
141             selected: ids.length,
142             total_sec: Math.round(sel.totalSec),
143             below_floor: warning?.atRisk.length ?? 0,
144         });
145         try {
146             const res = await compileReel(videoId, ids, filename, cid);
147             const url = highlightUrl(videoId, filename);
148             setBuild({ status: "done", url, filename });
149             log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
150         } catch (err) {
151             // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads
"is the engine
152             // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500"
(A5).
153             const status = err instanceof ApiError ? err.status : undefined;
154             const e = classifyError(err);
155             setBuild({ status: "error", error: errorText(t, e) });
156             log("error", "build.failed", { cid, video_id: videoId, filename, status, code:
e.code, detail: e.detail, error: String(err) });
157         }
158     };
159
160     // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
161     // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
162     // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
163     const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
164         const cid = newCorrelationId();
165         log("info", "query.parse", { cid, source, ...parseTrace(q) });
166
167         // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked

```

```

168 // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
169 if (q.unavailable.length > 0) {
170   log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
171 }
172
173 if (!q.recognized) {
174   log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
175   return; // empty / unparseable ? leave the user's curation untouched
176 }
177
178 try {
179   const { ordered, selectedIds } = runQuery(rallies, q);
180   setOrder(ordered);
181   sel.apply(selectedIds);
182   log("info", "query.apply", {
183     cid,
184     source,
185     filters: q.filters.length,
186     sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
187     limit: q.limit ?? null,
188     total: rallies.length,
189     selected: selectedIds.length,
190     selected_ids: selectedIds,
191     dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
192   });
193   if (selectedIds.length === 0) {
194     // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
195     log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
196   }
197 } catch (err) {
198   // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
199   log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
200 }
201 };
202
203 // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
204 const onToggle = (id: number) => {
205   log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
206   sel.toggle(id);
207 };
208 const onBulk = (op: "select_all" | "clear" | "invert") => {
209   const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
210   log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
211   if (op === "select_all") sel.selectAll();
212   else if (op === "clear") sel.clearAll();
213   else sel.invert();
214 };
215
216 return (
217   <main className="min-h-screen bg-paper text-ink pb-24">
218     <header className="bg-ink text-white">
219       <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
220         <span className="text-xl font-extrabold tracking-tight">
221           Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
222         </span>

```

```

223         <LanguageSwitcher />
224     </div>
225 </header>
226
227     <section className="mx-auto max-w-5xl px-6 py-10">
228         <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
229             {t("app.badge")}
230         </span>
231         <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
232         <p className="mt-2 max-w-2xl text-ink/70">
233             <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
234         </p>
235
236         {/* Front-half ingest (B-P2): when no video is loaded, point the tool at a
bucket URI / host
237             path ? process ? load. Once a video_id exists, the existing back-half takes
over. */}
238         {!videoId && (
239             <IngestPanel
240                 onLoad={onLoaded} => {
241                     setVideoId(id);
242                     window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
243                 }
244             />
245         )}
246
247         <ModeBanner load={loadState} videoId={videoId} />
248
249         <div className="mt-6">
250             <QueryBar onApply={onApplyQuery} />
251             {/* A9: localized tap controls for users who can't type the English query
syntax. */}
252             <TapFilters onApply={onApplyQuery} />
253         </div>
254
255         <div className="mt-6 flex flex-wrap gap-2 text-sm">
256             <button type="button" onClick={() => onBulk("select_all")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.selectAll")}</button>
257             <button type="button" onClick={() => onBulk("clear")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.clear")}</button>
258             <button type="button" onClick={() => onBulk("invert")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.invert")}</button>
259         </div>
260
261         <div className="mt-5">
262             <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
263         </div>
264
265         <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
266     </section>
267
268     <SelectionFooter
269         count={sel.count}
270         totalSec={sel.totalSec}
271         reelName={reelName}
272         onReelNameChange={setReelName}
273         onBuild={onBuild}
274         building={build.status === "building"}
275         warning={warning}

```

```

276         />
277     </main>
278 );
279 }
280
281 function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
282     if (load.status === "demo") {
283         return (
284             <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
285                 <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
286             </p>
287         );
288     }
289     if (load.status === "loading") {
290         return (
291             <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
292                 <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code /> }} />
293             </p>
294         );
295     }
296     if (load.status === "error") {
297         return (
298             <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
299 text-red-800" data-testid="mode-banner">
300                 <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
301 components={{ code: <code /> }} />
302             </p>
303         );
304     }
305     return (
306         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
307             <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
308 <strong className="text-ink/70" /> }} />
309         </p>
310     );
311 }

```

6. assistant (2026-06-21T17:17:54.686Z)

Now let me verify the i18n catalogs have the tap.* blocks and check the test that pins tap==type.

7. user (2026-06-21T17:17:55.829Z)

```

web\src\lib\selection.test.ts
web\src\lib\query.test.ts
web\src\lib\floor.test.ts
web\src\lib\format.test.ts
web\src\components\QueryBar.examples.test.ts
web\src\hooks\useIngestJob.test.ts
web\src\lib\errors.test.ts
web\src\components\IngestPanel.test.tsx
web\src\components\A4.touch.test.tsx
web\src\components\build-error.test.tsx
web\src\lib\errorText.test.ts
web\src\components\A8.status.test.tsx
web\src\api\client.test.ts
web\src\api\upload.test.ts
web\src\components\upload.test.tsx
web\src\hooks\useUpload.test.ts
web\src\i18n\i18n.test.tsx
web\src\i18n\locales\en\common.json
web\src\i18n\locales\kn\common.json

```

```
web\src\i18n\locales\hi\common.json
web\src\i18n\locales\es\common.json
web\src\i18n\locales\da\common.json
web\src\i18n\locales\id\common.json
web\src\i18n\hygiene.test.ts
web\src\components\TapFilters.test.tsx
```

8. user (2026-06-21T17:17:57.012Z)

Found 4 files

```
web\src\components\TapFilters.test.tsx
web\src\i18n\hygiene.test.ts
web\src\App.tsx
web\src\components\TapFilters.tsx
```

9. user (2026-06-21T17:18:00.720Z)

```
1      // @vitest-environment jsdom
2      //
3      // A9 ? localized tap-only filters. The load-bearing guarantee: each tap selects
IDENTICALLY to
4      // typing its English equivalent (tap == type), so a Kannada/Hindi/? user who taps gets
exactly what
5      // an English user who types gets ? the parser stays the single source of truth.
6      import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";
7      import { render, screen, cleanup, fireEvent } from "@testing-library/react";
8      import i18n from "../i18n";
9      import { TapFilters, TAP_FILTERS } from "../TapFilters";
10     import { parseQuery } from "../lib/query";
11
12     beforeEach(async () => {
13       await i18n.changeLanguage("en");
14     });
15     afterEach(() => cleanup());
16
17     describe("TapFilters (A9)", () => {
18       it("each tap builds the SAME query as typing its English equivalent (tap == type)", ()
=> {
19         for (const f of TAP_FILTERS) {
20           const typed = parseQuery(f.english);
21           expect(typed.recognized, f.english).toBe(true);
22           expect(typed.unavailable, f.english).toEqual([]); // length-led, fully available ?
never shot-count
23           expect({ filters: typed.filters, sort: typed.sort, limit: typed.limit },
f.english).toEqual({
24             filters: f.parts.filters,
25             sort: f.parts.sort,
26             limit: f.parts.limit,
27           });
28         }
29       });
30
31       it("tapping a control applies a recognized query through onApply", () => {
32         const onApply = vi.fn();
33         render(<TapFilters onApply={onApply} />);
34         fireEvent.click(screen.getByRole("button", { name: "Over 10 seconds" }));
35         expect(onApply).toHaveBeenCalledTimes(1);
36         const [q, source] = onApply.mock.calls[0];
37         expect(source).toBe("example");
38         expect(q.recognized).toBe(true);
39         expect(q.filters).toEqual([{ field: "duration", op: ">", value: 10 }]);
40         expect(q.unavailable).toEqual([]);
41       });
42
43       it("renders localized control labels (Kannada)", async () => {
```

```

44     await i18n.changeLanguage("kn");
45     render(<TapFilters onApply={() => {}} />);
46     expect(screen.getByText("??? ?????? ?????")).toBeTruthy(); // tap.longest ? Kannada
47     await i18n.changeLanguage("en");
48   });
49
50   it("every control is a 744px tap target (A4)", () => {
51     render(<TapFilters onApply={() => {}} />);
52     for (const f of TAP_FILTERS) {
53       expect(screen.getByText(i18n.t(f.key)).className, f.key).toMatch(/min-h-11/);
54     }
55   });
56 });
57

```

10. user (2026-06-21T17:18:01.644Z)

```

web\src\i18n\locales\da\common.json:101: "tap": {
web\src\i18n\locales\da\common.json:102:   "title": "Eller tryk for at filtrere:",
web\src\i18n\locales\da\common.json:103:   "longest": "Længste først",
web\src\i18n\locales\da\common.json:104:   "shortest": "Korteste først",
web\src\i18n\locales\da\common.json:105:   "over10": "Over 10 sekunder",
web\src\i18n\locales\da\common.json:106:   "over20": "Over 20 sekunder",
web\src\i18n\locales\da\common.json:107:   "top5": "De 5 længste",
web\src\i18n\locales\da\common.json:108:   "top10": "De 10 længste"
C:\Users\avidu\Projects\kheIsutra\web\src\i18n\locales\da\common.json:109- },
web\src\i18n\locales\da\common.json:110:   "selection": {
web\src\i18n\locales\da\common.json:111:     "count": "{count, plural, one {# duel valgt} other
{# dueller valgt}}"
--
web\src\i18n\locales\kn\common.json:101: "tap": {
web\src\i18n\locales\kn\common.json:102:   "title": "???? ??????? ?????? ?????? ????:",
web\src\i18n\locales\kn\common.json:103:   "longest": "??? ??????? ??????",
web\src\i18n\locales\kn\common.json:104:   "shortest": "??? ??????? ??????",
web\src\i18n\locales\kn\common.json:105:   "over10": "10 ????????????? ??????",
web\src\i18n\locales\kn\common.json:106:   "over20": "20 ????????????? ??????",
web\src\i18n\locales\kn\common.json:107:   "top5": "??? ????? 5",
web\src\i18n\locales\kn\common.json:108:   "top10": "??? ????? 10"
C:\Users\avidu\Projects\kheIsutra\web\src\i18n\locales\kn\common.json:109- },
web\src\i18n\locales\kn\common.json:110:   "selection": {
web\src\i18n\locales\kn\common.json:111:     "count": "{count, plural, one {# ?????? ?????????????}
other {# ?????????? ?????????????}}"
--
web\src\i18n\locales\es\common.json:101: "tap": {
web\src\i18n\locales\es\common.json:102:   "title": "0 toca para filtrar:",
web\src\i18n\locales\es\common.json:103:   "longest": "Las más largas primero",
web\src\i18n\locales\es\common.json:104:   "shortest": "Las más cortas primero",
web\src\i18n\locales\es\common.json:105:   "over10": "Más de 10 segundos",
web\src\i18n\locales\es\common.json:106:   "over20": "Más de 20 segundos",
web\src\i18n\locales\es\common.json:107:   "top5": "Las 5 más largas",
web\src\i18n\locales\es\common.json:108:   "top10": "Las 10 más largas"
C:\Users\avidu\Projects\kheIsutra\web\src\i18n\locales\es\common.json:109- },
web\src\i18n\locales\es\common.json:110:   "selection": {
web\src\i18n\locales\es\common.json:111:     "count": "{count, plural, one {# jugada
seleccionada} many {# jugadas seleccionadas} other {# jugadas seleccionadas}}"
--
web\src\i18n\locales\id\common.json:101: "tap": {
web\src\i18n\locales\id\common.json:102:   "title": "Atau ketuk untuk memfilter:",
web\src\i18n\locales\id\common.json:103:   "longest": "Terpanjang dulu",
web\src\i18n\locales\id\common.json:104:   "shortest": "Terpendek dulu",
web\src\i18n\locales\id\common.json:105:   "over10": "Lebih dari 10 detik",
web\src\i18n\locales\id\common.json:106:   "over20": "Lebih dari 20 detik",
web\src\i18n\locales\id\common.json:107:   "top5": "5 terpanjang",
web\src\i18n\locales\id\common.json:108:   "top10": "10 terpanjang"
C:\Users\avidu\Projects\kheIsutra\web\src\i18n\locales\id\common.json:109- },

```



```

web\src\i18n\locales\id\common.json-110- "selection": {
web\src\i18n\locales\id\common.json-111-   "count": "{count, plural, other {# reli dipilih}}"
--
web\src\i18n\locales\en\common.json:101- "tap": {
web\src\i18n\locales\en\common.json-102-   "title": "Or tap to filter:",
web\src\i18n\locales\en\common.json-103-   "longest": "Longest first",
web\src\i18n\locales\en\common.json-104-   "shortest": "Shortest first",
web\src\i18n\locales\en\common.json-105-   "over10": "Over 10 seconds",
web\src\i18n\locales\en\common.json-106-   "over20": "Over 20 seconds",
web\src\i18n\locales\en\common.json-107-   "top5": "Top 5 longest",
web\src\i18n\locales\en\common.json-108-   "top10": "Top 10 longest"
C:\Users\avidu\Projects\kheIsutra\web\src\i18n\locales\en\common.json-109- },
web\src\i18n\locales\en\common.json-110- "selection": {
web\src\i18n\locales\en\common.json-111-   "count": "{count, plural, one {# rally selected}
other {# rallies selected}}"
--
web\src\i18n\locales\hi\common.json:101- "tap": {
web\src\i18n\locales\hi\common.json-102-   "title": "?? ?????? ??? ? ? ? ? ??:",
web\src\i18n\locales\hi\common.json-103-   "longest": "???? ???? ????",
web\src\i18n\locales\hi\common.json-104-   "shortest": "???? ???? ????",
web\src\i18n\locales\hi\common.json-105-   "over10": "10 ????? ? ? ??????",
web\src\i18n\locales\hi\common.json-106-   "over20": "20 ????? ? ? ??????",
web\src\i18n\locales\hi\common.json-107-   "top5": "???? ???? ? 5",
web\src\i18n\locales\hi\common.json-108-   "top10": "???? ???? ? 10"
C:\Users\avidu\Projects\kheIsutra\web\src\i18n\locales\hi\common.json-109- },
web\src\i18n\locales\hi\common.json-110- "selection": {
web\src\i18n\locales\hi\common.json-111-   "count": "{count, plural, one {# ???? ?????} other
{# ?????? ?????}}"

```

11. assistant (2026-06-21T17:18:18.854Z)

Now let me trace the parser carefully. Let me manually verify the tap==type invariant for each filter. The critical concern from the prompt is whether `parseQuery` produces exactly the same `{filters, sort, limit}` as the hardcoded `parts`.

Let me trace each case precisely.

```

***"longest first"*** (parts: {filters:[], sort:{duration,desc}}):
- LEADING/TRAILING comparator patterns: no number, so no filters. ?
- SORT_PATTERNS: `/\b(?:longest(?: first)?|...)\b/` matches "longest first" ? `{duration, desc}`. ?
- Limit: no top/first/last/N-longest match... wait. Let me check the limit regex
  `/\b(longest|shortest)\s+(\d+)\b` ? "longest first" has no digit, so no match. And
  `/\b(\d+)\b...\s+(longest|shortest)` ? no digit. So limit stays undefined. ?
- Result: `{filters:[], sort:{duration,desc}, limit:undefined}` matches parts. ?

***"shortest first"*** ? symmetric ? `{filters:[], sort:{duration,asc}, limit:undefined}`. ?

***"over 10s"*** (parts: {filters:[{duration,>,10}], no sort, no limit}):
- LEADING `lead("over|...", ">")`: regex
  `(?:over|...)\s*(\d+...)(?!d)(?:\s*(unit)\b)?(?:\s*UNAVAIL_WORD\b)`. "over 10s" ? number=10,
  unit="s". `makeFilter("10","s",>)` ? "s" is not `^shots?$/`, toSeconds(10,"s") = "s" doesn't
  match `^m/` ? 10. Filter `{duration,>,10}`. ?
- SORT: none match. ?
- Limit: none. ?
- Result matches. ?

***"over 20s"*** ? symmetric ? `{filters:[{duration,>,20}]}`. ?

***"top 5"*** (parts: {filters:[], sort:{duration,desc}, limit:5}):
- LEADING/TRAILING: "top 5" ? does any leading comparator match? No "over/under/..." word. No
  comparator. filters=[]. ?
- SORT_PATTERNS: does "top 5" match any sort regex? No. So sort stays undefined initially. ?
- Limit: the N-longest patterns don't match. Then `/\b(?:top|best)\s+(\d+)\b${NOT_UNIT}` ? "top
  5", NOT_UNIT = `(?!s*(?:shots?|seconds?|...)\b)` ? after "5" there's end of string, no unit ?

```

```

matches. limit=5. `!sort` true ? sort = `{duration, desc}`, sortFromLimit=true. ?
- limit validation: 5 is positive integer. ?
- Result: `{filters:[], sort:{duration,desc}, limit:5}` matches parts. ?

***top 10*** ? symmetric ? `{filters:[], sort:{duration,desc}, limit:10}`. ?

```

All six invariants hold. The test's deep-equal would catch drift. Now let me check the unavailable assertion and whether the test could miss a sort that the parts omit. In all cases the parts explicitly state the sort, and for over10/over20 the parts omit sort (`undefined`), and the parser produces `undefined` too. Good ? no drift the test would miss, because the test compares the full triple `{filters, sort, limit}`.

Let me verify the i18n hygiene test and check whether the QueryBar examples or another test references these. Let me also confirm App passes `onApplyQuery` and run the test suite.

12. user (2026-06-21T17:18:19.810Z)

```

1      // CI guardrails that keep i18n "by design" (FRIENDLY_AND_MULTILINGUAL.md §4):
2      // 1. KEY-PARITY ? every locale catalog has EXACTLY the `en` key set (the
extensibility proof:
3      //    drop a stub locale and this lists every gap). Trivially green while only `en`
exists; it
4      //    starts enforcing the moment a kn/hi catalog lands (B5/B6).
5      // 2. NO-BARE-STRINGS (attribute level, v1) ? user-facing component attributes
(placeholder /
6      //    aria-label / title) must route through t(), never a hardcoded literal. A curated
start per
7      //    the doc; an AST-level sweep of JSX text can extend it later.
8      import { describe, it, expect } from "vitest";
9      import { readdirSync, readFileSync } from "node:fs";
10     import { join } from "node:path";
11
12     const LOCALES_DIR = join(import.meta.dirname, "locales");
13     const SRC_DIR = join(import.meta.dirname, "..");
14
15     function flatten(obj: Record<string, unknown>, prefix = ""): string[] {
16       const keys: string[] = [];
17       for (const [k, v] of Object.entries(obj)) {
18         const key = prefix ? `${prefix}.${k}` : k;
19         if (v && typeof v === "object" && !Array.isArray(v)) keys.push(...flatten(v as
Record<string, unknown>, key));
20         else keys.push(key);
21       }
22       return keys.sort();
23     }
24
25     const loadKeys = (locale: string): string[] =>
26       flatten(JSON.parse(readFileSync(join(LOCALES_DIR, locale, "common.json"), "utf8")) as
Record<string, unknown>);
27
28     describe("i18n key-parity ? every locale has EXACTLY the en keys", () => {
29       const enKeys = loadKeys("en");
30       const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
31         .filter((d) => d.isDirectory() && d.name !== "en")
32         .map((d) => d.name);
33
34       it("en is the non-empty source catalog", () => {
35         expect(enKeys.length).toBeGreaterThan(20);
36       });
37
38       for (const loc of others) {
39         it(`${loc} matches en exactly (no missing, no extra keys)`, () => {
40           const keys = loadKeys(loc);
41           const missing = enKeys.filter((k) => !keys.includes(k));
42           const extra = keys.filter((k) => !enKeys.includes(k));

```

```

43         expect({ locale: loc, missing, extra }).toEqual({ locale: loc, missing: [], extra:
[] });
44     });
45 }
46 });
47
48 // Key-parity proves every locale has the same KEYS; this proves every locale's VALUE
keeps the same
49 // simple `{placeholder}` set as en ? so a translation can never silently drop a
{count}/{error}/{min}/
50 // {filename}/{n}/? and break interpolation. (ICU plural headers like `{count, plural,
?}` carry no
51 // simple {word} placeholder and match as empty on both sides, so they're correctly
ignored.)
52 describe("placeholder parity ? every locale preserves en's {placeholders}", () => {
53     function flatEntries(obj: Record<string, unknown>, prefix = ""): Record<string,
string> {
54         const out: Record<string, string> = {};
55         for (const [k, v] of Object.entries(obj)) {
56             const key = prefix ? `${prefix}.${k}` : k;
57             if (v && typeof v === "object" && !Array.isArray(v)) Object.assign(out,
flatEntries(v as Record<string, unknown>, key));
58             else out[key] = String(v);
59         }
60         return out;
61     }
62     const placeholders = (s: string) => (s.match(/\{(\w+)\}/g) ?? []).sort();
63     const load = (loc: string) => flatEntries(JSON.parse(readFileSync(join(LOCALES_DIR,
loc, "common.json"), "utf8")));
64     const enE = load("en");
65     const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
66         .filter((d) => d.isDirectory() && d.name !== "en")
67         .map((d) => d.name);
68
69     for (const loc of others) {
70         it(`${loc} preserves every {placeholder} that en uses`, () => {
71             const locE = load(loc);
72             const mismatches: Array<{ key: string; en: string[]; got: string[] }> = [];
73             for (const key of Object.keys(enE)) {
74                 if (!(key in locE)) continue; // missing keys are the key-parity test's job
75                 const a = placeholders(enE[key]);
76                 const b = placeholders(locE[key]);
77                 if (a.join(",") !== b.join(",")) mismatches.push({ key, en: a, got: b });
78             }
79             expect(mismatches).toEqual([]);
80         });
81     }
82 });
83
84 describe("es 3-form plural (CLDR one/many/other) ? MULTILINGUAL Phase A", () => {
85     // Spanish cardinal is {one, many, other}; every es ICU plural message MUST carry a
`many` arm
86     // (the `many` category fires on large/compact numbers) or intl-messageformat silently
falls back.
87     // These are the plural-bearing keys in the catalog.
88     const es = JSON.parse(readFileSync(join(LOCALES_DIR, "es", "common.json"), "utf8")) as
Record<string, Record<string, string>>;
89     const pluralKeys: Array<[string, string]> = [
90         ["footer", "count"],
91         ["footer", "floorSomeLead"],
92         ["selection", "count"],
93     ];
94     for (const [ns, key] of pluralKeys) {
95         it(`es ${ns}.${key} carries one/many/other arms`, () => {
96             const msg = es[ns][key];

```

```

97         expect(msg, `${ns}.${key}`).toMatch(/\\bone\\s*\\{/);
98         expect(msg, `${ns}.${key}`).toMatch(/\\bmany\\s*\\{/);
99         expect(msg, `${ns}.${key}`).toMatch(/\\bother\\s*\\{/);
100     });
101     }
102 });
103
104 describe("no-bare-strings ? user-facing component attributes go through t()", () => {
105     const FILES = [
106         "App.tsx",
107         "components/QueryBar.tsx",
108         "components/TapFilters.tsx",
109         "components/RallyGrid.tsx",
110         "components/SelectionFooter.tsx",
111         "components/CompileResult.tsx",
112         "components/LanguageSwitcher.tsx",
113         "components/IngestPanel.tsx",
114     ];
115     // A hardcoded placeholder / aria-label / title with letters is a bare-string
116     // must be an expression ({t(...)}), not a quoted literal.
117     const BARE_ATTR = /\\b(?:placeholder|aria-label|title)\\s*=\\s*"\\^[^"]*"\\s*\\/g;
118
119     for (const f of FILES) {
120         it(`${f}: no hardcoded placeholder/aria-label/title`, () => {
121             const src = readFileSync(join(SRC_DIR, f), "utf8");
122             expect(src.match(BARE_ATTR) ?? []).toEqual([]);
123         });
124     }
125 });
126
127 describe("Indic fonts (B4) ? self-hosted OFL Noto, no runtime CDN", () => {
128     it("main.tsx bundles the OFL Noto Kannada + Devanagari fonts (self-hosted, not Google Fonts)", () => {
129         const main = readFileSync(join(SRC_DIR, "main.tsx"), "utf8");
130         expect(main).toMatch(/@fontsource-variable\\/noto-sans-kannada/);
131         expect(main).toMatch(/@fontsource-variable\\/noto-sans-devanagari/);
132     });
133     it("the font stack falls back to Noto for Kannada + Devanagari glyphs", () => {
134         const css = readFileSync(join(SRC_DIR, "index.css"), "utf8");
135         expect(css).toMatch(/Noto Sans Kannada Variable/);
136         expect(css).toMatch(/Noto Sans Devanagari Variable/);
137     });
138 });
139
140 // The Studio SPA must render correctly on an AIR-GAPPED appliance (GPU/CPU/RAM, no
141 // internet) ?
142 // every asset (fonts, styles, scripts) is bundled local, nothing pulled from a
143 // CDN/Google Fonts.
144 describe("offline / self-contained ? no external runtime assets", () => {
145     const CDN = /googleapis|gstatic|fonts\\.google|cdnjs|unpkg|jsdelivr|bcdn\\. [a-z]/i;
146     const FILES = [
147         "../index.html", // web/index.html
148         "main.tsx",
149         "index.css",
150     ];
151     for (const rel of FILES) {
152         it(`${rel}: no external font/CDN reference`, () => {
153             expect(readFileSync(join(SRC_DIR, rel), "utf8")).not.toMatch(CDN);
154         });
155     }
156     it("web/index.html loads no absolute-URL <link>/<script src> (everything is
157     bundled/local)", () => {
158         const html = readFileSync(join(SRC_DIR, "..", "index.html"), "utf8");
159         expect(html).not.toMatch(/(?:href|src)\\s*=\\s*"\\s*https?:\\/\\/i);

```

```
157     });
158   });
159
```

13. user (2026-06-21T17:18:19.952Z)

```
1    // The fast-entry query bar (PER_RALLY_SELECTION.md P3). A deterministic, OFFLINE parser
turns
2    // plain text ("top 15 longest", "over 10s") into a selection over the SAME shared set
the
3    // checkboxes use. It is honest by construction: a live "parsed-pill" shows exactly how
the
4    // text was understood, and concepts with no data behind them (shot-type / player /
score) are
5    // greyed out as "not available yet" ? never silently faked.
6    import { useMemo, useState } from "react";
7    import type { FormEvent } from "react";
8    import { useTranslation, Trans } from "react-i18next";
9    import { parseQuery, summarize } from "../lib/query";
10   import type { ParsedQuery } from "../lib/query";
11
12   interface Props {
13     /** Apply a parsed query to the rally set (App runs runQuery + writes the shared
selection).
14     * `source` distinguishes a typed-and-submitted query from an example-chip tap (for
the trace). */
15     onApply: (q: ParsedQuery, source: "typed" | "example") => void;
16   }
17
18   // Each example is a real, parseable query ? tapping one fills the box AND applies it.
Led by rally
19   // LENGTH (the trusted axis); shot-count filtering exists but is experimental, so it's
not featured
20   // here (a test pins that every example parses to a recognized, fully-available query).
21   export const EXAMPLES = ["over 10s", "top 15 longest", "shortest first", "first 10"];
22
23   // Parsed-pill chips: legible on a phone (text-sm, not text-xs) ? A4.
24   const CHIP = "rounded-full px-3 py-1 text-sm font-medium";
25
26   export function QueryBar({ onApply }: Props) {
27     const { t } = useTranslation();
28     const [value, setValue] = useState("");
29     const parsed = useMemo(() => parseQuery(value), [value]);
30     const clauses = summarize(parsed);
31     const typed = value.trim().length > 0;
32
33     const applyText = (text: string) => {
34       setValue(text);
35       onApply(parseQuery(text), "example");
36     };
37     const submit = (e: FormEvent) => {
38       e.preventDefault();
39       if (parsed.recognized) onApply(parsed, "typed");
40     };
41
42     return (
43       <div>
44         <form onSubmit={submit} className="flex flex-wrap items-center gap-2">
45           <input
46             type="text"
47             value={value}
48             onChange={(e) => setValue(e.target.value)}
49             placeholder={t("query.placeholder")}
50             aria-label={t("query.ariaLabel")}
51             className="min-h-11 min-w-[16rem] flex-1 rounded-xl border border-black/15 bg-
```

```

white px-4 py-2 text-sm text-ink outline-none focus:border-green"
52         />
53         <button
54             type="submit"
55             disabled={!parsed.recognized}
56             className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
57         >
58             {t("query.apply")}
59         </button>
60     </form>
61
62     {/* Live interpretation ? transparent + hand-correctable (no hidden NL guessing).
*/}
63     <div aria-live="polite" data-testid="query-pill" className="mt-2 flex flex-wrap
items-center gap-1.5">
64         {clauses.map((c) => (
65             <span key={c} className={CHIP + " bg-green/15 text-green"}>
66                 {c}
67             </span>
68         ))}
69         {parsed.unavailable.map((u) => (
70             <span
71                 key={u.token}
72                 title={t("query.notAvailableTitle", { token: u.token, kind: u.kind })}
73                 className={CHIP + " bg-black/5 text-ink/45"}
74             >
75                 <span className="line-through">{u.token}</span> · {t("query.notAvailable")}
76             </span>
77         ))}
78         {typed && !parsed.recognized && parsed.unavailable.length === 0 && (
79             <span className="text-xs text-ink/50">{t("query.unreadable")}</span>
80         )}
81         {!typed && <span className="text-xs text-ink/50">{t("query.hint")}</span>}
82     </div>
83
84     {/* Surface the "not available yet" reason as VISIBLE text (the per-chip title is
hover-only,
85         so it never appears on a phone) ? A4. */}
86     {parsed.unavailable.length > 0 && (
87         <p className="mt-1.5 text-xs text-ink/50">{t("query.notAvailableHelp")}</p>
88     )}
89
90     <div className="mt-3 flex flex-wrap items-center gap-2 text-sm">
91         <span className="text-ink/40">{t("query.try")}</span>
92         {EXAMPLES.map((ex) => (
93             <button
94                 key={ex}
95                 type="button"
96                 onClick={() => applyText(ex)}
97                 className="inline-flex min-h-11 items-center rounded-full border border-
black/15 px-4 text-sm hover:border-green"
98             >
99                 {ex}
100             </button>
101         ))}
102     </div>
103
104     <p className="mt-2 text-xs text-ink/40">
105         <Trans i18nKey="query.support" components={{ b: <strong className="text-ink/60"
/>, x: <span className="text-amber-700" /> }} />
106     </p>
107 </div>
108 );
109 }

```

14. assistant (2026-06-21T17:18:24.443Z)

Now let me verify the build/tests actually pass and that there's no drift. Let me run the targeted tests for TapFilters and the parser, plus tsc.

15. user (2026-06-21T17:18:33.280Z)

```
RUN v3.2.6 C:/Users/avidu/Projects/khelsutra/web

? src/i18n/hygiene.test.ts (28 tests)
16ms
? src/lib/query.test.ts (46 tests) 25ms
(node:43184) ExperimentalWarning: localStorage is not available because --localstorage-file was
not provided.
(Use `node --trace-warnings ...` to show where the warning was created)
? src/components/TapFilters.test.tsx (4 tests)
227ms

Test Files  3 passed (3)
Tests      78 passed (78)
Start at   22:48:29
Duration   3.48s (transform 188ms, setup 39ms, collect 635ms, tests 268ms,
environment 2.01s, prepare 968ms)
```

16. assistant (2026-06-21T17:18:37.932Z)

Tests pass. Let me run the full web suite and tsc to confirm the "173 web tests green; tsc+build clean" claim, and double-check the runQuery behavior with the constructed query (specifically the `raw: "tap:tap.longest"` string flowing through parseTrace/logging).

17. user (2026-06-21T17:18:56.329Z)

```
stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-21T17:18:51.449Z","level":"error","event":"api.network_error","cid":"f88c428c-1ec3-47ea-af2e-5e0132813d69","method":"GET","url":"/api/config","error":"Error: no engine in test"}

stdout | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-21T17:18:51.475Z","level":"info","event":"api.request","cid":"9d3969fe-3c9e-4b01-8019-574bd5041aa9","method":"GET","url":"/api/config"}

stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-21T17:18:51.488Z","level":"error","event":"api.network_error","cid":"9d3969fe-3c9e-4b01-8019-574bd5041aa9","method":"GET","url":"/api/config","error":"Error: no engine in test"}
{"ts":"2026-06-21T17:18:51.488Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-21T17:18:51.488Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}

stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-21T17:18:51.489Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error calling GET /api/config"}
```

```

? src/i18n/i18n.test.tsx (17 tests)
1131ms
? i18n scaffold (B1) > renders user-facing copy from the en
catalog (not raw keys) 529ms

Test Files 19 passed (19)
Tests 173 passed (173)
Start at 22:48:43
Duration 7.84s (transform 2.29s, setup 323ms, collect 9.00s, tests 5.03s,
environment 45.18s, prepare 8.88s)

=== TSC ===
TSC EXIT: 0

```

18. assistant (2026-06-21T17:19:11.923Z)

173 tests green, tsc clean. Now let me address the three review questions precisely.

Question 1 ? tap==type invariant. I traced all six manually above; all hold. The test does a full deep-equal on `{filters, sort, limit}`, so it would catch any drift including "parseQuery adds a sort the parts omit" (for over10/over20, parts omit sort ? `undefined`, and the parser also yields `undefined` since no SORT_PATTERN matches and no limit triggers `sortFromLimit`).

Question 2 ? onApplyQuery handling. The constructed query has `recognized: true` so it passes the `!q.recognized` guard, flows to `runQuery` + `sel.apply`. The `raw`: `"tap:tap.longest"` string is purely a log/trace value ? it's never re-parsed (the component builds the `ParsedQuery` directly). `parseTrace` just echoes `raw` into the `query.parse` log line. No issue. Let me confirm one subtle thing about `runQuery` with the over10/over20 case ? those have no `sort`, so `runQuery` uses `rallies.slice()` for ordered (input order preserved), then filters. That's identical to a typed "over 10s". Good.

Question 3 ? length-led + fully-available. All six filters operate on `duration` only (sort key `duration`, filter field `duration`). None use `shot_count`. The test asserts `typed.unavailable` equals `[]` for each. No shot-type/player/score tokens. Confirmed.

Let me do one final precise check: confirm the constructed query object shape exactly matches `ParsedQuery` (no missing field that App relies on) ? App reads `q.raw`, `q.recognized`, `q.unavailable`, `q.filters`, `q.sort`, `q.limit`. The constructed object provides all of these. For over10/over20, `sort` and `limit` are spread from `f.parts.sort`/`f.parts.limit` which are `undefined` ? that matches a parser output where those are `undefined`. Good.

One thing worth verifying: does the `key` field of the catalog match? The component calls `t(f.key)` where `f.key = "tap.longest"`, and the catalog nests under `tap.longest`. The Kannada test confirms `t("tap.longest")` resolves. And `t("tap.title")` resolves to the title. Confirmed across all six locales (longest/shortest/over10/over20/top5/top10 + title all present).

Everything checks out. Let me verify the git diff scope to ensure nothing unexpected is bundled, then finalize.

19. user (2026-06-21T17:19:13.759Z)

```

web/src/App.tsx | 3 ++
web/src/components/TapFilters.test.tsx | 56 +++++
web/src/components/TapFilters.tsx | 58 +++++
web/src/i18n/hygiene.test.ts | 1 +
web/src/i18n/locales/da/common.json | 9 +++++
web/src/i18n/locales/en/common.json | 9 +++++
web/src/i18n/locales/es/common.json | 9 +++++
web/src/i18n/locales/hi/common.json | 9 +++++
web/src/i18n/locales/id/common.json | 9 +++++
web/src/i18n/locales/kn/common.json | 9 +++++
10 files changed, 172 insertions(+)
=== branch ===
feat/b3-site-i18n

```


20. assistant (2026-06-21T17:19:18.321Z)

The diff is clean and scoped exactly to A9: the new component + test, the App wiring (3 lines), the hygiene test entry adding TapFilters.tsx to the no-bare-strings sweep, and the `tap.\*` block across all six locales. No unrelated changes.

I've verified everything end to end. Let me record the findings.

21. user (2026-06-21T17:19:26.223Z)

Structured output provided successfully

22. assistant (2026-06-21T17:19:40.221Z)

A9 is correct ? no findings. Summary of what I verified:

```
**1. tap==type invariant (all six hold).** I traced each control through `parseQuery`:
- `longest first` ? `{filters:[], sort:{duration,desc}}`, no limit (no digit, so the N-longest
limit regex never fires).
- `shortest first` ? `{filters:[], sort:{duration,asc}}`.
- `over 10s` / `over 20s` ? `{filters:[{duration,>,10|20}]}`, **no sort, no limit** ?
matches the parts which omit both.
- `top 5` / `top 10` ? no comparator, no SORT_PATTERN match, then the `top|best\s+(\d+)`
limit branch sets `limit=5|10` and (since `!sort`) `sort={duration,desc}` with
`sortFromLimit=true`; validation passes (positive integer).
```

All match `TAP\_FILTERS[].parts` exactly. The pinning test deep-equals the full
`{filters,sort,limit}` triple **and** asserts `unavailable==[]`, so it would catch any drift ?
including "parser adds a sort the parts omit" (verified the over10/over20 case yields
`sort:undefined` on both sides, since nothing triggers a phantom limit-implied sort).

```
**2. App.onApplyQuery handling is identical to a typed query.** The constructed object
(`recognized:true`, `unavailable:[]`, `filters/sort/limit` from parts) passes the
`!q.recognized` guard, runs `runQuery` + `setOrder` + `sel.apply`, and logs
`query.parse`/`query.apply` the same way. The `raw:"tap:tap.longest"` string is only echoed into
log lines via `parseTrace` ? it is never re-parsed (the component builds the ParsedQuery
directly), so there's no risk of it being misinterpreted.
```

```
**3. Length-led + fully-available.** Every control uses `duration` only (sort key and filter
field); none touch `shot_count`, none carry unavailable tokens.
```

Verification run locally: full web suite **173/173 green** (TapFilters 4, hygiene 28, query 46 among them), `tsc --noEmit` exits 0. The `tap.\*` block (title + longest/shortest/over10/over20/top5/top10) is present in all six catalogs, with key-parity enforced by the hygiene test. Diff is scoped cleanly to A9.

Relevant files:

- C:/Users/avidu/Projects/khelsutra/web/src/components/TapFilters.tsx
- C:/Users/avidu/Projects/khelsutra/web/src/components/TapFilters.test.tsx
- C:/Users/avidu/Projects/khelsutra/web/src/lib/query.ts
- C:/Users/avidu/Projects/khelsutra/web/src/App.tsx